

PRODUCT REQUIREMENTS DOCUMENT

PantryPilot

Code-grounded as-built specification for the safety-aware meal and grocery planning demonstration

Document status	As-built specification
Backend baseline	v1.1.0 - Safety & Grocery Controls
Web layer	Unreleased
Owner	Zixuan Liang
Version and date	PRD 1.0 July 21, 2026
Source	PantryPilot_CM_Generic_Files code, tests, configuration, and release records

1. Purpose

This product requirements document reverse-engineers PantryPilot from the implemented repository. It defines the behavior that exists in the current executable baseline, the acceptance evidence that verifies that behavior, and the product concepts that remain incomplete or out of scope.

The document is intentionally conservative. A configuration flag, screen label, or future concept is not treated as a completed capability unless the code exposes the behavior and tests or repeatable system scenarios verify it.

2. Executive Summary

PantryPilot is a local, safety-aware meal and grocery planning demonstration. It presents three planned meals and lets a household user evaluate recipe safety against declared allergens, consolidate ingredients into a grocery list, classify pantry items by expiry proximity, and inspect release-quality evidence.

The product's central quality principle is:

Candidate recipe or optimization output must not become user-facing advice until deterministic controls validate it against the household profile.

The current backend baseline is version 1.1.0. The Flask web presentation layer is recorded under the repository's Unreleased section. The solution is designed as an executable course-project baseline, not as a production multi-user service.

3. Product Context

3.1 Problem Statement

Household meal planning often requires users to move between recipes, dietary constraints, pantry inventory, and shopping lists. This creates three risks:

- A recipe can violate a declared allergen or dietary constraint.
- Duplicate ingredients can produce inconsistent or inflated shopping quantities.
- Pantry expiry boundaries can be interpreted incorrectly, causing waste or unsafe recommendations.

PantryPilot demonstrates how deterministic controls and release evidence can reduce those risks before more complex AI or optimization features are introduced.

3.2 Product Vision

PantryPilot should become a unified household planning assistant that can propose meals, validate safety constraints, account for pantry inventory, and produce a reliable grocery plan. The current implementation establishes a small, testable foundation for that larger vision.

3.3 Current Product State

The current application:

- Runs locally through a Flask server bound to `127.0.0.1`.
- Uses three in-memory recipes and five in-memory pantry items.
- Loads release and quality-gate configuration from version-controlled JSON.
- Performs deterministic allergen, grocery, and expiry calculations in Python.
- Exposes the same behavior through JSON endpoints and a responsive browser UI.
- Executes eleven pytest tests and six formal release-level system scenarios.
- Stores no user profile, plan, grocery list, or pantry changes after a browser refresh.

4. Goals and Success Criteria

4.1 Product Goals

1. Prevent a known allergen alias from being approved for a profile that declares that allergen.
2. Demonstrate that a safety decision comes from deterministic backend logic rather than browser-only presentation code.
3. Consolidate grocery items by normalized name and unit.

4. Apply an explicit, inclusive pantry expiry boundary.
5. Make release identity, required quality gates, and system-test evidence visible in the application.
6. Keep the demonstration local and repeatable without a database or external service.

4.2 Release Success Criteria

The current release scope is accepted when all of the following are true:

- A recipe containing soy sauce is blocked when the profile declares soy.
- The same recipe is approved when no declared allergen alias is present.
- Three olive oil entries of 1 tbsp consolidate to 3 tbsp.
- Two tomato entries of 2 each consolidate to 4 each.
- An item expiring today or at plus three days is classified as use-soon for a three-day window.
- An item at plus four days is not use-soon for a three-day window.
- An already expired item is classified separately and excluded from use-soon counts.
- The Quality view reports all four configured gates and six of six passing release scenarios.
- All eleven pytest tests pass.

4.3 Non-Goals for the Current Release

The current implementation does not attempt to provide:

- User authentication, authorization, or multi-user tenancy.
- Persistent profiles, plans, grocery lists, or pantry inventory.
- Recipe create, update, delete, search, or import workflows.
- A live large-language-model or AI recipe-generation service.
- Nutrition optimization or constraint solving.
- Pantry-driven meal recommendation or automatic plan modification.
- Unit conversion across grocery items with different units.
- Production deployment, telemetry, alerting, or service-level objectives.
- Payment, ordering, delivery, notification, or calendar integrations.

5. Users and Stakeholders

5.1 Household Planner

The primary demonstration user wants to review meals, declare allergens, produce a consolidated grocery list, and identify pantry items that should be used soon.

5.2 Project Reviewer

A course instructor, reviewer, or tester wants to inspect the controlled release identity, confirm quality gates, execute repeatable scenarios, and compare visible behavior with documented results.

5.3 Product Owner and Developer

The individual project owner maintains the code, risk register, tests, configuration items, release records, and acceptance evidence.

6. Scope Assessment

Table 1. *Current capability scope*

Capability	Current status	Code-grounded interpretation
Household allergen profile	Implemented for demo	Four checkbox options are held in browser memory; Soy is selected by default.
Recipe library	Partial	Three hard-coded recipes are displayed; there is no storage, search, or recipe CRUD.
Weekly meal planning	Partial	Three dated meal cards are displayed; users cannot add, remove, reorder, or save meals.
Allergen safety gate	Implemented	Backend alias matching returns approved or blocked decisions.
Grocery generation	Implemented	Selected demo recipes are consolidated by normalized item name and unit.
Pantry expiry status	Implemented	Five demo items are classified as fresh, use-soon, or expired.
Pantry-aware planning	Not implemented	Pantry status does not change meal selection or grocery quantities.
AI substitutions	Not implemented in executable code	The feature flag is enabled, but no model call, candidate generator, or substitution workflow exists.
Nutrition optimization	Not implemented	The feature flag is disabled and no optimizer exists.
Quality evidence	Implemented	The UI displays configured gates and executes six release-level scenarios.
Data persistence	Not implemented	Application and browser state reset on refresh or restart.

7. User Experience Requirements

7.1 Primary Navigation

The application shall provide four main views: Plan, Grocery, Pantry, and Quality. Navigation shall use buttons that update the current view without requiring a full page reload.

7.2 Plan Workflow

The Plan view shall display three planned dinners, summary metrics, household allergen controls, and a safety result for the selected recipe. The default state shall select the Ginger tofu rice bowl and declare Soy as an active allergen, producing one blocked recipe.

7.3 Grocery Workflow

The Grocery view shall allow the user to include or exclude any of the three demo recipes and generate one consolidated ingredient table. The page shall report both the number of selected recipes and the number of canonical grocery items.

7.4 Pantry Workflow

The Pantry view shall display the fixed demonstration date, item expiry states, use-soon and expired counts, and selectable one-, three-, and five-day windows.

7.5 Quality Workflow

The Quality view shall display the release identity, four configured quality gates, a control to run release checks, the expected and actual result of each scenario, and an aggregate pass count.

7.6 Reset Workflow

The reset control shall restore the default Soy allergy, selected tofu recipe, all grocery recipes selected, a three-day expiry window, an unexecuted Quality view, and the Plan view as the current destination.

8. Functional Requirements

8.1 Release and Demo Data

Table 2. 1 Release and Demo Data: requirements and acceptance criteria

ID	Requirement	Acceptance criteria
FR-001	The system shall load release configuration from config/app_config.json.	The returned identity is PantryPilot 1.1.0: Safety & Grocery Controls.
FR-002	The system shall provide the UI with release metadata, feature flags, quality gates, the default profile, recipes, pantry items, and the demo date.	GET /api/demo-data returns HTTP 200, release version 1.1.0, three recipes, five pantry items, and demo date 2026-08-14.
FR-003	The UI shall display the controlled baseline version in the navigation area.	The sidebar displays v1.1.0 after initialization.

8.2 Recipe Safety

Table 3. 2 Recipe Safety: requirements and acceptance criteria

ID	Requirement	Acceptance criteria
FR-004	The system shall normalize ingredient and allergen terms by trimming whitespace and converting values to lowercase.	Rice and rice normalize to the same token; blank values are ignored.
FR-005	The system shall detect conflicts using the configured deterministic alias map for peanut, soy, milk, and egg.	soy sauce and soy lecithin conflict with a declared soy allergen.
FR-006	The recipe-check API shall return the recipe identity, sorted conflict list, Boolean safety value, and approved or blocked decision.	A request for tofu-bowl with soy returns safe: false, conflicts: ["soy"], and decision: "blocked".
FR-007	The UI shall reevaluate all demo recipes whenever the user changes an allergen checkbox.	Clearing Soy changes the tofu recipe from blocked to approved; restoring Soy blocks it again.
FR-008	The browser shall present backend safety results and shall not independently calculate recipe safety.	static/app.js obtains decisions from POST /api/check-recipe; the deterministic implementation remains in src/pantrypilot_app.py.

8.3 Grocery Aggregation

Table 4. 3 Grocery Aggregation: requirements and acceptance criteria

ID	Requirement	Acceptance criteria
FR-009	The user shall be able to choose which demo recipes contribute to the grocery list.	The Grocery view exposes one checkbox per recipe and submits the selected IDs.
FR-010	The system shall aggregate grocery quantities by lowercase, trimmed item name and lowercase, trimmed unit.	Three olive oil records using tbsp produce one 3.0 tbsp record.
FR-011	The system shall keep items with different units as different canonical records.	Aggregation keys contain both item name and unit; no unit conversion occurs.
FR-012	The grocery API shall return the consolidated items, selected recipe count, and item count.	Selecting all demo recipes returns three recipes, sixteen canonical items, olive oil: 3.0 tbsp, and tomato: 4.0 each.
FR-013	An empty recipe selection shall produce an empty list rather than a fabricated item.	The response contains zero recipes and zero items, and the UI displays its empty-selection message.

8.4 Pantry Expiry

Table 5. 4 Pantry Expiry: requirements and acceptance criteria

ID	Requirement	Acceptance criteria
FR-014	The system shall parse pantry dates in ISO YYYY-MM-DD format.	Valid ISO dates are accepted; invalid dates return HTTP 400 from the pantry API.

ID	Requirement	Acceptance criteria
FR-015	The use-soon rule shall include day zero through the selected future window, inclusive.	For August 14 with a three-day window, August 14 and August 17 are use-soon; August 18 is fresh.
FR-016	The system shall classify dates before the current date as expired rather than use-soon.	The August 13 lemon is expired when the demo date is August 14.
FR-017	The pantry API shall accept window values from 1 through 14 days.	Values below 1 or above 14 return HTTP 400.
FR-018	The UI shall expose one-, three-, and five-day window controls and update the displayed classifications.	Changing from three to five days increases the demo use-soon count from two to three.

8.5 Quality Evidence

Table 6. 5 Quality Evidence: requirements and acceptance criteria

ID	Requirement	Acceptance criteria
FR-019	The system shall load and expose the configured unit-test, pull-request, review, and CI gates.	All four gates are present and enabled in the current configuration and Quality view.
FR-020	The Quality endpoint shall execute the formal system-test runner when requested.	GET /api/quality-summary invokes run_system_tests() and returns six scenario records.
FR-021	Each system scenario shall calculate PASS or FAIL by comparing its actual value with an expected value.	No scenario receives a hard-coded display status.
FR-022	The Quality view shall display each scenario ID, description, actual result, and status.	After execution, six scenario rows and a 6 / 6 score are visible.
FR-023	The system shall report an overall passed status only when every returned scenario passes.	status is passed when passed == total; the current result is six of six.

8.6 Input Validation and Errors

Table 7. 6 Input Validation and Errors: requirements and acceptance criteria

ID	Requirement	Acceptance criteria
FR-024	The recipe API shall reject an unknown recipe ID.	The endpoint returns HTTP 404 and Unknown recipe..
FR-025	The recipe API shall require allergens to be a list of strings.	Invalid allergen payloads return HTTP 400.
FR-026	The grocery API shall require recipe IDs to be a list and reject unknown IDs.	A non-list returns HTTP 400; any unknown ID returns HTTP 404.
FR-027	The pantry API shall reject invalid dates, nonnumeric windows, and windows outside 1-14.	Each invalid request returns HTTP 400 with a concise error message.
FR-028	The UI shall present initialization failure text if startup data cannot be loaded.	The loading screen changes to an error state containing the returned message.

9. Business Rules

BR-001: Allergen Alias Rule

The current alias map covers:

- Peanut: peanut, peanuts, peanut butter, groundnut.
- Soy: soy, soybean, soybeans, soy sauce, soy lecithin.
- Milk: milk, dairy, butter, cream, cheese.
- Egg: egg, eggs, mayonnaise.

Matching is exact after trimming and lowercasing. The current control does not parse free-form ingredient descriptions, detect misspellings, infer cross-contamination, or consult manufacturer data.

BR-002: Grocery Canonicalization Rule

Items are grouped by the tuple (normalized name, normalized unit). Quantities are converted to floating-point numbers and summed. The result is sorted by name and unit. The system does not convert between cups, ounces, tablespoons, grams, or item counts.

BR-003: Expiry Boundary Rule

An item is use-soon when:

$$0 \leq \text{expiration date} - \text{current date} \leq \text{window days}$$

A negative difference is expired. A positive difference beyond the window is fresh.

BR-004: Release Status Rule

Version 1.1.0 is the tagged backend baseline. The Flask UI and its web tests are maintained under Unreleased until they pass the established branch, review, CI, changelog, and tagging process.

10. API Contracts

Table 8. HTTP API contracts

Method and path	Purpose	Primary request	Successful response	Error behavior
GET /	Load the browser application.	None	Rendered index.html.	Standard Flask error behavior.
GET /api/demo-data	Initialize release, profile, recipe, pantry, and gate data.	None	JSON demonstration payload.	Configuration or server failures surface as HTTP errors.
POST /api/check-recipe	Evaluate one recipe against declared allergens.	recipe_id, allergens[]	Recipe identity, conflicts, safe flag, decision.	400 for invalid allergens; 404 for unknown recipe.
POST /api/grocery-list	Consolidate selected recipe ingredients.	recipe_ids[]	Items, recipe count, item count.	400 for non-list; 404 for unknown recipe IDs.

Method and path	Purpose	Primary request	Successful response	Error behavior
POST /api/pantry-status	Classify pantry items for a date and window.	today, window_days	Item statuses and summary counts.	400 for invalid date, type, or range.
GET /api/quality-summary	Run formal release scenarios.	None	Release, gates, scenarios, totals, overall status.	Server error if configuration or test execution fails.

11. Data Requirements

11.1 Recipe

A demo recipe contains id, name, description, day, duration, servings, image path, tags, safety-check ingredients, and structured grocery items. Recipe IDs shall be unique within the in-memory collection.

11.2 Grocery Item

A grocery item contains name, numeric quantity, and unit. The normalized name and unit form its aggregation key.

11.3 Pantry Item

A pantry item contains id, name, display quantity, and ISO expiry date. The API adds days_remaining and status to the response.

11.4 Profile

The demo profile contains a display name and an allergen list. The browser permits Soy, Peanut, Milk, and Egg selections. Profile state is not persisted.

11.5 Release Configuration

The version-controlled configuration contains application identity, baseline version, release name, environment, feature flags, and quality-gate flags.

12. Nonfunctional Requirements

Table 9. *Nonfunctional requirements and current evidence*

ID	Requirement	Current evidence or limitation
NFR-001	Safety and boundary decisions shall be deterministic for identical inputs.	Pure Python functions and repeatable tests produce stable results.
NFR-002	The demo shall function without a database or external application API.	Data and images are local; Lucide assets are stored in the repository.
NFR-003	The local server shall not expose itself beyond the loopback interface by default.	Flask binds to 127.0.0.1. This is not a production security control.

ID	Requirement	Current evidence or limitation
NFR-004	The browser layout shall adapt from desktop to small-screen widths.	CSS breakpoints exist at 1040, 820, and 560 pixels, with a 320-pixel minimum layout target.
NFR-005	Interactive controls shall be keyboard-focusable and carry useful semantic labels.	Native buttons and inputs, focus-visible styles, landmarks, labels, and live regions are present. Formal WCAG conformance has not been audited.
NFR-006	The implementation shall be testable without launching a browser.	Flask's test client verifies routes and response contracts.
NFR-007	CI shall run on each relevant push and pull request to main.	GitHub Actions uses Python 3.12 and runs pytest plus system_test_runner.py.
NFR-008	Runtime dependencies shall be pinned for reproducibility.	Flask 3.1.1 and pytest 8.2.2 are pinned in requirements.txt. Transitive packages are not locked.
NFR-009	The demo shall provide a repeatable starting state.	The reset action restores the documented browser state.
NFR-010	The release shall have explicit test evidence.	Eleven pytest tests and six formal scenarios currently pass. No latency, load, or availability SLO is defined.

13. Acceptance Test Scenarios

AT-01: Default Safety Block

Given the default Soy allergen and selected tofu bowl, **when** the Plan view finishes loading, **then** the tofu recipe is blocked, the conflict list contains Soy, and the weekly summary reports one safety conflict.

AT-02: Profile-Driven Reevaluation

Given the tofu bowl is blocked, **when** the user clears Soy, **then** the recipe becomes approved. **When** Soy is selected again, **then** the recipe returns to blocked.

AT-03: Consolidated Grocery List

Given all three recipes are selected, **when** the user generates the grocery list, **then** the response contains sixteen canonical items, including olive oil: 3 tbsp and tomato: 4 each.

AT-04: Inclusive Pantry Window

Given the date August 14, 2026 and a three-day window, **when** pantry status is calculated, **then** Broccoli and Tofu are use-soon, Spinach is fresh, and Lemon is expired. **When** the window changes to five days, **then** Spinach becomes use-soon and the use-soon count becomes three.

AT-05: Release Quality Summary

Given the Quality view is ready, **when** the user runs release checks, **then** all four configured gates are visible, six scenario records are returned, and the score is six of six passed.

AT-06: Invalid API Input

Given an invalid recipe ID, malformed allergen list, malformed recipe ID list, invalid date, or out-of-range expiry window, **when** the relevant endpoint receives the request, **then** it returns a 400 or 404 response and does not fabricate a successful result.

14. Requirements Traceability

Table 10. *Requirements-to-evidence traceability*

Evidence	Requirements covered	Repository source
Release identity test	FR-001 to FR-003	tests/test_smoke.py
Quality-gate test	FR-019	tests/test_smoke.py
Allergen block and safe-recipe tests	FR-004 to FR-008	tests/test_smoke.py
Grocery aggregation test	FR-009 to FR-013	tests/test_smoke.py
Expiry boundary test	FR-014 to FR-018	tests/test_smoke.py
Demo page and release-data test	FR-002, FR-003	tests/test_web_app.py
Recipe endpoint test	FR-006	tests/test_web_app.py
Grocery endpoint test	FR-010, FR-012	tests/test_web_app.py
Pantry endpoint test	FR-015, FR-016, FR-018	tests/test_web_app.py
Quality endpoint test	FR-020 to FR-023	tests/test_web_app.py
Six formal system scenarios	Release, gates, grocery, safety, and boundary behavior	system_test_runner.py
Responsive browser review	NFR-004, NFR-005	templates/index.html, static/styles.css
CI workflow	NFR-007, NFR-010	.github/workflows/pantrypilot-ci.yml

15. Configuration and Release Requirements

All source, tests, configuration, web assets, requirements, CI files, change records, and release notes are controlled configuration items. Changes shall follow the documented workflow:

1. Create a scoped branch from main.
2. Update implementation, tests, configuration, and documentation together.
3. Run pytest and the formal system-test runner locally.
4. Open a pull request and review functional and configuration impact.
5. Require successful CI before merge.
6. Update the changelog and release notes.
7. Create a version tag only after release acceptance.

16. Risks and Product Limitations

16.1 Safety Coverage

The allergen control uses a small exact-match alias dictionary. It can miss composite ingredient text, spelling variants, manufacturer warnings, cross-contamination statements, and allergens outside the four declared categories. It is evidence of a deterministic control pattern, not a substitute for clinical or regulatory food-safety validation.

16.2 Demo Data and Persistence

Recipes, pantry items, profile state, and the demo date are hard-coded. No edits survive refresh or restart. This supports repeatability but prevents real household use.

16.3 UI and Backend Consistency

The backend is the source of truth for safety, grocery, pantry, and quality results. Endpoint tests reduce contract risk, but automated browser end-to-end testing is not implemented.

16.4 Configuration Semantics

The `ai_substitutions` flag is true even though no AI generation or substitution code exists. Product status should be represented by both configuration and executable acceptance evidence before future releases.

16.5 Production Readiness

The application has no authentication, persistent storage, rate limiting, audit log, secret-management model, monitoring, backup, recovery, or production deployment design. It shall remain a local demonstration until those requirements are specified and verified.

17. Recommended Roadmap

Release 1.2: Formalize the Web Baseline

- Move the Flask UI from Unreleased into a reviewed and tagged baseline.
- Add automated tests for every documented 400 and 404 response.
- Add browser-level end-to-end tests for the four primary workflows and reset behavior.
- Add an accessibility audit and correct identified issues.
- Reconcile feature flags with implemented capability status.

Release 2.0: Persistent Household Planning

- Add authenticated household profiles and persistent allergen preferences.

- Add recipe and pantry create, update, delete, search, and import workflows.
- Save weekly plans and grocery-list completion state.
- Define data migration, privacy, backup, and audit requirements.

Later Releases: Controlled Intelligence

- Introduce an AI candidate-generation interface behind the deterministic safety gate.
- Add model and prompt versioning, offline fixtures, adversarial safety cases, and fallback behavior.
- Add nutrition optimization only after feasibility, timeout, relaxation, and explainability requirements are defined.
- Use pantry inventory to modify recommendations and grocery quantities only after unit conversion and freshness rules are verified.

18. Open Decisions

1. Should the web layer become version 1.2.0 or be incorporated into a revised 1.1.x baseline?
2. Should `ai_substitutions` remain enabled before an executable substitution workflow exists?
3. What authoritative allergen taxonomy and ingredient parser should replace the demo alias map?
4. What persistence model and privacy controls are required for household profile data?
5. Which unit-conversion rules are acceptable for grocery aggregation?
6. What performance, availability, security, and accessibility targets are required before production deployment?
7. Which browser and mobile device matrix should become part of release acceptance?

Appendix A: Code Sources Used

- `src/pantrypilot_app.py`: deterministic release, safety, aggregation, and expiry functions.
- `src/web_app.py`: demo data, Flask routes, validation, and quality summary.
- `templates/index.html`: Plan, Grocery, Pantry, and Quality views.
- `static/app.js`: browser state, API integration, rendering, navigation, and reset behavior.
- `static/styles.css`: visual presentation, focus styles, and responsive behavior.
- `config/app_config.json`: release identity, feature flags, and quality gates.
- `tests/test_smoke.py`: six core regression tests.
- `tests/test_web_app.py`: five Flask route and integration tests.
- `system_test_runner.py`: six formal release-level scenarios.
- `requirements.txt`: pinned direct dependencies.
- `CHANGELOG.md` and `releases/`: status accounting and release records.
- `.github/workflows/pantrypilot-ci.yml`: automated CI execution.

Appendix B: Verification Snapshot

At the time this PRD was generated:

- Pytest result: 11 passed.
- Formal system scenarios: 6 passed, 0 failed.
- Backend baseline: v1.1.0.
- Flask demonstration status: Unreleased.
- Local demonstration address: `http://127.0.0.1:5050` when started with `PORT=5050 .venv/bin/python -m src.web_app`.